

Documentación de uso de IA — Prompts y mejoras

1. Revisión inicial del proyecto

Prompt utilizado

“Según la consigna del proyecto, revisar todos los archivos. Si hay alguna modificación indícamelo y explícamelo de forma sencilla como para principiantes. Prioriza que el botón principal y la generación de paletas funcionen correctamente. Implementa primero el caso más simple y luego agrega mejoras. Prueba frecuentemente tu aplicación en el navegador. Realiza commits pequeños y descriptivos. Revisa que la app funcione sin errores. Verifica que el responsive y la accesibilidad básica estén contemplados”.

¿Para qué nos ayudó?

Este fue el punto de partida para revisar el proyecto completo.

Se analizaron los tres archivos principales:

index.html

styles.css

app.js

La prioridad fue comprobar que:

- El botón **"Generar paleta"** funcionara.
- Se pudieran generar diferentes cantidades de colores.
- Los formatos **HSL y RGB/HEX** funcionaran.
- Los colores bloqueados no cambiaran al generar una nueva paleta.
- Las paletas pudieran guardarse.
- El historial utilizara localStorage.
- La aplicación fuera responsive.
- Existieran elementos básicos de accesibilidad.

Resultado: se mantuvo una estructura sencilla y se fueron agregando mejoras sin modificar innecesariamente el funcionamiento principal.

2. Revisión y documentación del proyecto

Prompt utilizado

“Quiero ahora generar el README con los siguientes puntos: Asegúrate de que el README explique claramente cómo usar y desplegar el proyecto”.

¿Para qué nos ayudó?

Se planteó la estructura de documentación del proyecto para explicar:

- Qué hace la aplicación.
- Cómo utilizarla.
- Cómo ejecutarla.
- Cómo desplegarla.
- Qué tecnologías utiliza.
- Cómo está organizado el proyecto.

Esto permitió que el repositorio no tuviera solamente código, sino también instrucciones para que otra persona pudiera entenderlo y ejecutarlo.

3. Mejora del botón RGB en HTML

Uno de los cambios solicitados fue revisar la opción correspondiente al formato RGB.

La parte importante del HTML quedó estructurada de esta manera:

```
HTML

<label>
  <input
    type="radio"
    name="color-format"
    value="rgb"
  >
  RGB / HEX
</label>
```

¿Para qué nos ayudó?

El usuario puede seleccionar claramente:

HSL

RGB / HEX

Y el JavaScript utiliza ese valor para decidir qué tipo de color generar.

Esto mantiene separadas las responsabilidades:

- **HTML:** ofrece la opción.
- **CSS:** define su apariencia.
- **JavaScript:** decide qué hacer cuando se selecciona.

4. Mejora del botón "Desbloqueado"

Problema detectado

Cuando se generaban **9 colores**, el texto del botón:

 Desbloqueado

podía ocupar demasiado espacio dentro de las tarjetas.

Prompt utilizado

“Quisiera agregar una mejora recomendada, solo quedar en el HTML en el input RGB. Lo que quisiera ver es en qué parte del proyecto puedo hacer que esté centrada la descripción de cada paleta "Desbloqueado" cuando se generan 9 paletas, porque al momento de generarlo esta cantidad el texto rebasa el espacio establecido”.

Solución

La modificación principal se realizó en styles.css, específicamente en `”.lock-button”`.

Se agregaron propiedades como:

```
(/) CSS

text-align: center;
white-space: normal;
overflow-wrap: anywhere;
```

¿Qué conseguimos?

El texto del botón puede:

- mantenerse centrado;
- ocupar varias líneas si es necesario;
- adaptarse al espacio disponible;

- evitar que el texto se salga de la tarjeta.

Esto es especialmente útil cuando la pantalla muestra **9 colores**.

5. Mejora visual: mensaje "Copiar"

Después se planteó una mejora para que el usuario supiera que podía hacer clic sobre el código del color.

Prompt utilizado

“Otro punto que agregar, que al momento de pasar el mouse por la paleta tenga un mensaje instructivo de "copiar", qué códigos tengo que adicionar y que sean códigos fáciles de entender como para principiantes, siguiendo la línea del proyecto y que no altere mucho el actual”.

Solución

Se modificó `.color-code` para permitir colocar un mensaje mediante un pseudo-elemento.

Se agregó:

```
</> CSS
```

```
.color-code {  
  position: relative;  
}
```

Y posteriormente:

```
css
.color-code::after {
  content: "Copiar";

  position: absolute;

  left: 50%;
  bottom: 100%;

  transform: translateX(-50%);

  margin-bottom: 6px;

  padding: 5px 8px;

  color: #ffffff;
  background: #222222;

  border-radius: 5px;

  opacity: 0;

  pointer-events: none;

  white-space: nowrap;
}
```

Después se indicó cuándo mostrarlo:

```
css
.color-code:hover::after {
  opacity: 1;
}
```

¿Para qué nos ayudó?

Antes el usuario tenía que descubrir por sí mismo que el código era clickeable.

Ahora, al colocar el mouse sobre el código aparece:

“Copiar”

Esto mejora la **usabilidad** porque la interfaz explica qué puede hacer el usuario.

6. Mejora de accesibilidad

Posteriormente se agregó también:

```
CSS

.color-code:focus-visible::after {
  opacity: 1;
}
```

¿Por qué fue importante?

No dependemos solamente del mouse.

Cuando el usuario navega utilizando el teclado y llega al botón del código, también puede visualizar la indicación:

“Copiar”

Esto complementa las reglas existentes:

```
CSS

.color-code:focus-visible {
  outline: 3px solid var(--color-foco);
  outline-offset: 2px;
}
```

Resultado

La aplicación contempla dos formas de interacción:

```
Mouse → hover → "Copiar"

Teclado → focus → "Copiar"
```

7. Revisión de app.js

Una parte importante del proyecto fue comprobar que las mejoras visuales no rompieran la funcionalidad existente.

El funcionamiento principal de app.js quedó basado en estas partes:

```
Generar color
↓
Crear tarjeta
↓
Mostrar código
↓
Permitir copiar
↓
Bloquear / desbloquear
↓
Generar nuevamente
↓
Mantener colores bloqueados
```

La función principal continúa siendo:

```
// JavaScript
function generarPaleta() {
```

Esta obtiene la cantidad seleccionada:

```
// JavaScript
const cantidad = Number(paletteSize.value);
```

Y el formato:

```
// JavaScript
const formatoSeleccionado =
  document.querySelector(
    'input[name="color-format"]:checked'
  );
```

Después utiliza:

```
// JavaScript

const color = crearColor(formato);
```

para crear los colores.

8. Bloqueo de colores

Uno de los puntos funcionales más importantes fue mantener los colores bloqueados.

En app.js se utiliza:

```
// JavaScript

tarjeta.dataset.bloqueado = "true";
```

Cuando se genera una nueva paleta se comprueba:

```
// JavaScript

const estaBloqueado =
  tarjeta.dataset.bloqueado === "true";

if (estaBloqueado) {
  continue;
}
```

¿Para qué sirve?

Si el usuario hace:

```
Generar paleta
↓
Bloquea un color
↓
Generar paleta nuevamente
```

el color bloqueado permanece igual.

Los demás colores sí cambian.

Esto es una de las funcionalidades principales de la aplicación.

9. Copiar los códigos

Otra funcionalidad importante que revisamos fue la copia del código.

En app.js existe:

```
</> JavaScript
async function copiarCodigo(codigo) {
```

y utiliza:

```
</> JavaScript
await navigator.clipboard.writeText(codigo);
```

Después informa al usuario:

```
</> JavaScript
mostrarMensaje(`${codigo} copiado al portapapeles`);
```

Flujo

```
graph TD
    A[Usuario hace clic en el código] --> B[copiarCodigo()]
    B --> C[navigator.clipboard]
    C --> D[Código copiado]
    D --> E[Aparece mensaje]
```

La mejora visual de "Copiar" realizada en CSS simplemente **explica esta función existente**, sin modificar innecesariamente el JavaScript.

10. Mensajes al usuario

También revisamos la sección de mensajes:

```
</> JavaScript
function mostrarMensaje(mensaje) {
```

Esta función utiliza el elemento:

```
JavaScript
const toast = document.querySelector("#toast");
```

Y muestra temporalmente mensajes como:

```
¡Nueva paleta generada!
Color bloqueado
Color desbloqueado
¡Paleta guardada!
Paleta eliminada
Código copiado al portapapeles
```

El mensaje desaparece después de unos segundos mediante:

```
JavaScript
setTimeout(() => {
  toast.classList.remove("visible");
}, 3000);
```

¿Para qué nos ayudó?

El usuario recibe una confirmación después de realizar una acción.

Esto evita que tenga que adivinar si la operación funcionó.

11. Guardado e historial

Otra funcionalidad importante revisada fue el historial.

La aplicación utiliza:

```
JavaScript
const STORAGE_KEY = "paletasGuardadas";
```

y:

```
// JavaScript

localStorage.setItem(
  STORAGE_KEY,
  JSON.stringify(paletasGuardadas)
);
```

Esto permite guardar las paletas en el navegador.

El flujo es:

```
Generar paleta
↓
Guardar paleta
↓
localStorage
↓
Historial
↓
Mostrar paletas guardadas
```

También existe la posibilidad de eliminar una paleta:

```
// JavaScript

eliminarPaleta(indice);
```

12. Responsive

Otra parte importante de la revisión fue comprobar el comportamiento en diferentes tamaños de pantalla.

Se mantuvieron tres situaciones principales:

```
Pantalla grande
↓
hasta 1200 px aproximadamente

Tablet
↓
max-width: 900px

Móvil
↓
max-width: 600px
```

Por ejemplo, en móvil la paleta utiliza:

```
css
.palette {
  grid-template-columns:
    repeat(2, minmax(0, 1fr));
}
```

Mientras que en tablet:

```
css
.palette {
  grid-template-columns:
    repeat(4, 1fr);
}
```

¿Para qué nos ayudó?

La aplicación puede utilizarse desde:

- computadora;
- tablet;
- teléfono.

Además, el botón principal ocupa todo el ancho disponible en móvil:

```
css
#generate-button {
  width: 100%;
}
```

13. Accesibilidad básica

Durante la revisión también se conservaron elementos como:

```
html
aria-label
aria-live
aria-describedby
aria-labelledby
```

Por ejemplo:

```
</> HTML

<p
  id="color-count"
  class="color-count"
  aria-live="polite">
</p>
```

Esto permite comunicar cambios dinámicos al usuario.

También se mantienen indicadores de foco:

```
</> CSS

button:focus-visible,
select:focus-visible,
input:focus-visible {
  outline: 3px solid var(--color-foco);
}
```

14. Estructura final del proyecto

Para tu documentación puedes representar el proyecto de esta manera:

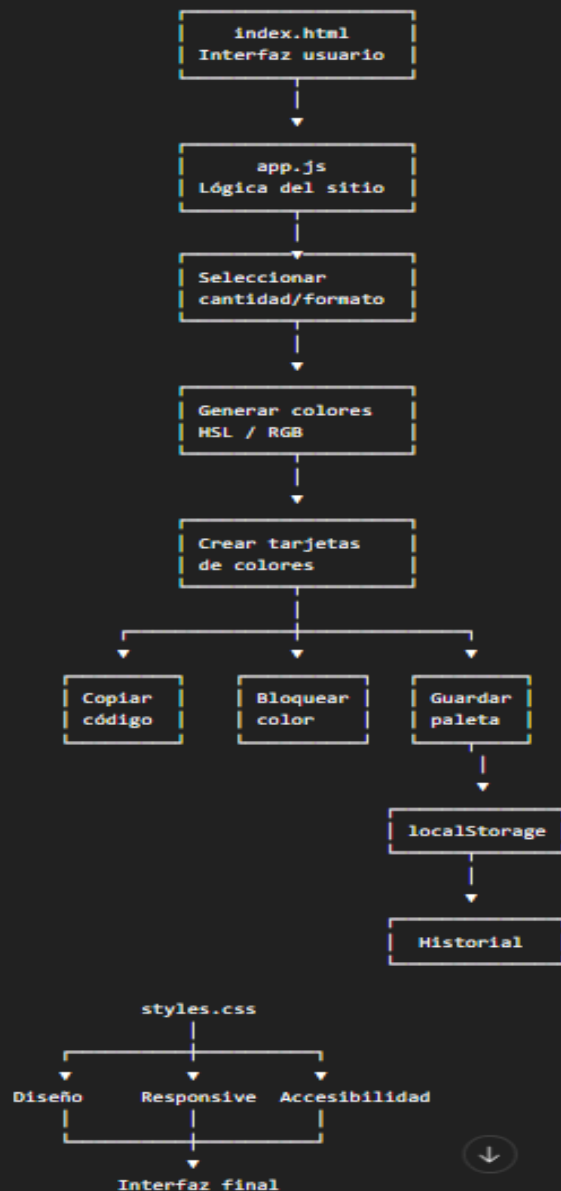
```
Proyecto-M1_MauricioLopez/
├── index.html
├── styles.css
├── app.js
├── README.md
├── Documentación/
│   ├── capturas/
│   │   ├── inicio.png
│   │   ├── paleta-generada.png
│   │   ├── color-bloqueado.png
│   │   ├── copiar-color.png
│   │   ├── historial.png
│   │   └── responsive.png
│   ├── uso-ia/
│   │   └── prompts-y-mejoras.md
│   └── README.pdf
```

Si tu consigna exige específicamente documentación de IA con capturas, puedes añadir:

```
Documentación/  
└─ uso-ia/  
    │ prompts-y-mejoras.md  
    │ captura-01-revision.png  
    │ captura-02-html.png  
    │ captura-03-css.png  
    └─ captura-04-javascript.png
```

Flujo general de nuestro proyecto

Este es el flujo que te recomiendo incluir también en la documentación:



Resumen de la interacción con IA:

Durante el desarrollo del proyecto se utilizó la IA como herramienta de apoyo para revisar el código HTML, CSS y JavaScript, detectar posibles problemas y proponer mejoras sencillas. Primero se priorizó el funcionamiento principal del generador de paletas. Posteriormente se realizaron mejoras de usabilidad, como el ajuste del botón "Desbloqueado", el mensaje "Copiar" al pasar sobre los códigos de color y su adaptación al uso mediante teclado. También se revisaron aspectos de responsive, accesibilidad, almacenamiento de paletas e historial. Los cambios fueron aplicados de manera progresiva, manteniendo la estructura original del proyecto y procurando que cada modificación fuera fácil de comprender.